

Universidad Anáhuac México

Facultad de Ingeniería

Guía de Ejemplos

Algoritmos y Programación en Python

Nivel Básico — Primer Semestre

Cinco ejemplos (A–E) con formato enunciado / solución

- **A:** Ciclos `for` anidados — patrones y combinaciones
- **B:** Módulo `random` — `randint` y `choice`
- **C:** Ordenamiento — `.sort()` y `sorted()`
- **D:** Ciclos `while` y `while True`
- **E:** Sentencia `match/case` básica

Prof. DSc. Aboud Barsekh-Onji

Facultad de Ingeniería | Universidad Anáhuac México

Ciclo escolar 2025-2026

0 Índice

1. Ejemplo A — Ciclos for anidados	2
2. Ejemplo B — Módulo random	5
3. Ejemplo C — Ordenamiento de listas	8
4. Ejemplo D — Ciclos while y while True	11
5. Ejemplo E — Sentencia match/case	14
Resumen comparativo	17

1 Ejemplo A — Ciclos **for** anidados

Objetivo

Practicar el uso de dos ciclos **for** anidados para imprimir patrones en pantalla y generar combinaciones a partir de dos listas.

Conceptos necesarios

- **for variable in range(n):** — repite el bloque **n** veces.
- **Ciclo externo:** se mueve una vez por fila (como el horario del reloj).
- **Ciclo interno:** completa *todas* sus iteraciones por cada iteración del externo (como el minutero del reloj).
- **print(..., end=' ')** — imprime sin saltar de línea.
- **print()** — imprime solo el salto de línea.
- **for elemento in lista:** — recorre cada elemento de una lista.

Enunciado — `ejemploA.py`

Escribe un programa con tres partes que usen ciclos **for** anidados:

1. **Rectángulo de asteriscos.** Imprime en pantalla un rectángulo de **4 filas** por **5 columnas** de asteriscos. Usa un ciclo **for** externo para las filas y uno interno para las columnas. Al terminar cada fila, imprime un salto de línea con **print()**.
2. **Tablas de multiplicar del 1, 2 y 3.** Usando dos ciclos **for** anidados, imprime las tablas del **1 al 3** (con multiplicadores del 1 al 5). El ciclo externo debe recorrer las tablas (1, 2, 3); el interno, los multiplicadores (1 al 5). Deja una línea en blanco entre cada tabla.
3. **Combinaciones de frutas y colores.** Dadas las listas:

```
frutas = ['manzana', 'pera', 'uva']  
colores = ['rojo', 'verde']
```

Usa dos ciclos **for** anidados para imprimir *todas* las combinaciones posibles de fruta + color (p. ej. **manzana - rojo**, **manzana - verde**, etc.).

Solución — Ejemplo A: Ciclos **for** anidados

ejemploA.py

```

1 # =====
2 # ejemploA.py - Ciclos FOR anidados (nivel básico)
3 # Materia: Algoritmos y Programación en Python
4 # Prof. Dr. Aboud Barsekh-Onji | Universidad Anáhuac México
5 # =====
6
7 # --- Parte 1: Rectángulo de asteriscos ---
8 print("=" * 35)
9 print(" RECTÁNGULO DE ASTERISCOS (4 x 5)")
10 print("=" * 35)
11
12 for fila in range(4):          # 4 filas
13     for columna in range(5):    # 5 columnas por fila
14         print("*", end=" ")
15     print()                    # salto de línea al terminar cada fila
16
17 # --- Parte 2: Tabla de multiplicar del 1 al 3 ---
18 print("\n" + "=" * 35)
19 print(" TABLAS DEL 1, 2 Y 3 (del 1 al 5)")
20 print("=" * 35)
21
22 for tabla in range(1, 4):       # tablas: 1, 2, 3
23     for mult in range(1, 6):    # multiplicadores: 1 al 5
24         print(f" {tabla} x {mult} = {tabla * mult}")
25     print()                    # línea en blanco entre tablas
26
27 # --- Parte 3: Combinaciones de frutas y colores ---
28 print("=" * 35)
29 print(" COMBINACIONES: FRUTA + COLOR")
30 print("=" * 35)
31
32 frutas = ["manzana", "pera", "uva"]
33 colores = ["rojo", "verde"]
34
35 for fruta in frutas:
36     for color in colores:
37         print(f" {fruta} - {color}")

```

Listing 1: ejemploA.py — For anidados (básico)

Parte 1 — Rectángulo de asteriscos. El ciclo exterior recorre las filas; por cada fila, el ciclo interior imprime 5 asteriscos con `end=' '` (sin saltar de línea). Al finalizar el ciclo interior, el `print()` vacío produce el salto de línea que termina la fila. Total de iteraciones: $4 \times 5 = 20$.

Parte 2 — Tablas de multiplicar. El ciclo exterior toma el número de tabla (`tabla`: 1, 2, 3). El ciclo interior hace el conteo de multiplicadores (`mult`: 1 a 5). El `print()` entre tablas añade una línea en blanco para separar visualmente.

Parte 3 — Combinaciones. El ciclo exterior recorre **frutas** (3 elementos); el interior recorre **colores** (2 elementos) completo para cada fruta. Resultado: $3 \times 2 = 6$ combinaciones impresas.

Salida esperada

```
* * * * *
* * * * *
* * * * *
* * * * *

1 x 1 = 1
1 x 2 = 2 ... 1 x 5 = 5

2 x 1 = 2 ... 3 x 5 = 15

manzana - rojo
manzana - verde
pera - rojo
pera - verde
uva - rojo
uva - verde
```

2 Ejemplo B — Módulo `random`

Objetivo

Aprender a generar números enteros aleatorios y a elegir elementos al azar de una lista usando las dos funciones esenciales del módulo `random`: `randint()` y `choice()`.

Conceptos necesarios

- `import random` — importar el módulo antes de usarlo.
- `random.randint(a, b)` — devuelve un entero aleatorio entre `a` y `b` inclusive (ambos extremos incluidos).
- `random.choice(lista)` — devuelve *un* elemento elegido al azar de la lista; la lista no se modifica.
- Acumulador: variable que suma resultados dentro de un ciclo.
- Condicional `if/else` dentro de un `for`.

Nota: los resultados varían en cada ejecución porque son aleatorios. Eso es normal y es precisamente el objetivo del módulo `random`.

Enunciado — `ejemploB.py`

Escribe un programa con tres partes usando `random`:

1. **Dado de 6 caras.** Usa `random.randint(1, 6)` dentro de un ciclo `for` para simular **5 lanzamientos** de un dado. Imprime el número de lanzamiento y el resultado.
2. **Sorteo de ganador.** Dada la lista:

```
participantes = ['Ana', 'Luis', 'Sofia', 'Carlos',  
                'Maria']
```

Imprime la lista completa y luego usa `random.choice()` para elegir e imprimir *un* ganador al azar.

3. **Lanzamiento de moneda.** Usando `random.choice(['Cara', 'Sello'])` dentro de un ciclo `for` de 6 repeticiones:
 - Imprime el resultado de cada lanzamiento.
 - Cuenta cuántas veces salió 'Cara' y cuántas 'Sello' usando dos variables contadoras.
 - Al final imprime el total de cada resultado.

Solución — Ejemplo B: Módulo **random**

ejemploB.py

```
1 # =====
2 # ejemploB.py - Módulo RANDOM (nivel básico)
3 # Materia: Algoritmos y Programación en Python
4 # Prof. Dr. Aboud Barsekh-Onji | Universidad Anáhuac México
5 # =====
6
7 import random
8
9 # --- Parte 1: Lanzar un dado 5 veces ---
10 print("=" * 35)
11 print(" DADO DE 6 CARAS – 5 LANZAMIENTOS")
12 print("=" * 35)
13
14 for lanzamiento in range(1, 6):
15     resultado = random.randint(1, 6)
16     print(f" Lanzamiento {lanzamiento}: {resultado}")
17
18 # --- Parte 2: Elegir un ganador al azar ---
19 print("\n" + "=" * 35)
20 print(" SORTEO DE GANADOR")
21 print("=" * 35)
22
23 participantes = ["Ana", "Luis", "Sofia", "Carlos", "Maria"]
24 print(f" Participantes: {participantes}")
25
26 ganador = random.choice(participantes)
27 print(f" El ganador es: {ganador}")
28
29 # --- Parte 3: Lanzar una moneda 6 veces y contar ---
30 print("\n" + "=" * 35)
31 print(" MONEDA – 6 LANZAMIENTOS")
32 print("=" * 35)
33
34 caras = 0
35 sellos = 0
36
37 for i in range(1, 7):
38     resultado = random.choice(["Cara", "Sello"])
39     print(f" Lanzamiento {i}: {resultado}")
40     if resultado == "Cara":
41         caras += 1
42     else:
43         sellos += 1
44
45 print(f"\n Total caras: {caras}")
46 print(f" Total sellos: {sellos}")
```

Listing 2: ejemploB.py — random.randint y random.choice

Parte 1 — Dado. `randint(1, 6)` genera un entero en $[1, 6]$ con igual probabilidad para cada valor. Se llama *dentro* del ciclo para obtener un resultado distinto en cada iteración. Si se pusiera fuera del ciclo, todos los lanzamientos mostrarían el mismo número.

Parte 2 — Sorteo. `random.choice(lista)` selecciona un elemento sin modificar la lista original. No hace falta conocer los índices: la función se encarga de elegir uno al azar.

Parte 3 — Moneda y contadores. Las variables **caras** y **sellos** comienzan en 0 y se incrementan con `+= 1` cada vez que el resultado coincide. Al final, ambas suman 6 (el total de lanzamientos).

Salida esperada (ejemplo — los valores cambian en cada ejecución)

```
Lanzamiento 1: 4
Lanzamiento 2: 1
Lanzamiento 3: 6
Lanzamiento 4: 3
Lanzamiento 5: 5

Participantes: ['Ana', 'Luis', 'Sofia', 'Carlos', 'Maria']
El ganador es: Sofia

Lanzamiento 1: Cara
Lanzamiento 2: Sello
...
Total caras: 3
Total sellos: 3
```


3 Ejemplo C — Ordenamiento de listas

Objetivo

Ordenar listas de números y palabras usando `.sort()` y `sorted()`, y comprender la diferencia clave entre ambos métodos.

Conceptos necesarios

- `lista.sort()` — ordena la lista **en su lugar** (modifica el original). No devuelve nada (`None`).
- `lista.sort(reverse=True)` — ordena de mayor a menor.
- `sorted(lista)` — devuelve una **nueva** lista ordenada; el original **no** cambia.
- Diferencia clave:

<code>.sort()</code>	modifica la lista original
<code>sorted()</code>	crea una lista nueva

Enunciado — `ejemploC.py`

Escribe un programa con tres partes sobre ordenamiento:

1. **Ordenar calificaciones de forma ascendente con `.sort()`.** Dada la lista:

```
calificaciones = [75, 90, 60, 85, 70, 95, 65]
```

Imprime la lista original, luego aplica `.sort()` e imprime la lista ordenada de menor a mayor.

2. **Ordenar puntajes de forma descendente.** Dada la lista:

```
puntajes = [55, 88, 72, 99, 61, 80]
```

Aplica `.sort(reverse=True)` para ordenarla de mayor a menor. Imprime el original y el resultado.

3. **Ordenar nombres con `sorted()` sin modificar el original.** Dada la lista:

```
nombres = ['Carlos', 'Ana', 'Sofia', 'Luis', 'Maria',  
           'Beto']
```

Usa `sorted(nombres)` para crear una nueva lista ordenada alfabéticamente. Imprime: la lista original, la nueva lista ordenada, y confirma que el original **no** cambió.

Solución — Ejemplo C: Ordenamiento

ejemploC.py

```

1  # =====
2  # ejemploC.py - Ordenamiento de listas (nivel básico)
3  # Materia: Algoritmos y Programación en Python
4  # Prof. Dr. Aboud Barsekh-Onji | Universidad Anáhuac México
5  # =====
6
7  # --- Parte 1: .sort() - ordenar una lista de números ---
8  print("=" * 40)
9  print(" .sort() - NÚMEROS ASCENDENTE")
10 print("=" * 40)
11
12 calificaciones = [75, 90, 60, 85, 70, 95, 65]
13 print(f" Original: {calificaciones}")
14
15 calificaciones.sort() # modifica la lista en su lugar
16 print(f" Ascendente: {calificaciones}")
17
18 # --- Parte 2: .sort(reverse=True) - ordenar descendente ---
19 print("\n" + "=" * 40)
20 print(" .sort(reverse=True) - DESCENDENTE")
21 print("=" * 40)
22
23 puntajes = [55, 88, 72, 99, 61, 80]
24 print(f" Original: {puntajes}")
25
26 puntajes.sort(reverse=True)
27 print(f" Descendente: {puntajes}")
28
29 # --- Parte 3: sorted() - ordenar sin modificar el original ---
30 print("\n" + "=" * 40)
31 print(" sorted() - NOMBRES ALFABÉTICO")
32 print("=" * 40)
33
34 nombres = ["Carlos", "Ana", "Sofia", "Luis", "Maria", "Beto"]
35 print(f" Original: {nombres}")
36
37 nombres_ordenados = sorted(nombres) # NO modifica nombres
38 print(f" Ordenados: {nombres_ordenados}")
39 print(f" Original ok: {nombres}") # sigue igual

```

Listing 3: ejemploC.py — .sort() y sorted()

Parte 1 — .sort() ascendente. `calificaciones.sort()` reorganiza los elementos dentro de la misma lista. Después de llamarlo, la variable `calificaciones` apunta a la lista ya ordenada. Imprimir antes y después de `.sort()` muestra el cambio.

Parte 2 — .sort(reverse=True). El parámetro `reverse=True` invierte el criterio de ordenamiento (de mayor a menor). Por defecto `reverse=False`, es decir, ascendente.

Parte 3 — sorted() sin modificar el original. `sorted(nombres)` devuelve una lista *nueva* y deja `nombres` intacto. Por eso se necesita asignar el resultado: `nombres_ordenados`

`= sorted(nombres)`. La tercera impresión confirma que **nombres** no cambió.

Salida esperada

```
Original:    [75, 90, 60, 85, 70, 95, 65]
Ascendente:  [60, 65, 70, 75, 85, 90, 95]
```

```
Original:    [55, 88, 72, 99, 61, 80]
Descendente: [99, 88, 80, 72, 61, 55]
```

```
Original:    ['Carlos', 'Ana', 'Sofia', 'Luis', 'Maria', 'Beto']
Ordenados:   ['Ana', 'Beto', 'Carlos', 'Luis', 'Maria', 'Sofia']
Original ok: ['Carlos', 'Ana', 'Sofia', 'Luis', 'Maria', 'Beto']
```

4 Ejemplo D — Ciclos **while** y **while True**

Objetivo

Entender y aplicar el ciclo **while** con una condición explícita para contar y acumular, y el patrón **while True** con **break** para controlar la salida del ciclo manualmente.

Conceptos necesarios

- **while condicion:** — repite el bloque *mientras* la condición sea **True**. La condición se evalúa al inicio de cada iteración.
- **Regla de oro:** el cuerpo del ciclo debe modificar la condición, o el ciclo se repetirá infinitamente.
- **variable += 1** — forma corta de **variable = variable + 1**.
- **Acumulador:** variable que suma valores dentro del ciclo (**suma += numero**).
- **while True:** — ciclo infinito; solo termina con **break**.
- **break** — sale inmediatamente del ciclo.

Enunciado — **ejemploD.py**

Escribe un programa con tres partes:

1. **Contar del 1 al 5 con while.** Declara una variable **contador = 1**. Usa un ciclo **while** para imprimir el valor de **contador** mientras sea menor o igual a 5. No olvides incrementar el contador dentro del ciclo. Al salir del ciclo, imprime **'Ciclo terminado.'**
2. **Suma del 1 al 10 con while.** Declara **numero = 1** y **suma = 0**. Usa un ciclo **while** para sumar todos los enteros del 1 al 10. Al terminar, imprime el resultado total. (El resultado correcto es 55.)
3. **while True con break.** Declara **intentos = 0**. Usa **while True** para repetir un bloque que:
 - Incrementa **intentos** en 1.
 - Imprime **'Intento numero X'**.
 - Si **intentos** llega a 4, imprime un mensaje de límite y sale del ciclo con **break**.

Solución — Ejemplo D: while y while True

ejemploD.py

```
1 # =====
2 # ejemploD.py - Ciclos WHILE y WHILE TRUE (nivel básico)
3 # Materia: Algoritmos y Programación en Python
4 # Prof. Dr. Aboud Barsekh-Onji | Universidad Anáhuac México
5 # =====
6
7 # --- Parte 1: while con contador simple ---
8 print("=" * 35)
9 print(" WHILE: CONTAR DEL 1 AL 5")
10 print("=" * 35)
11
12 contador = 1
13 while contador <= 5:
14     print(f" Contador: {contador}")
15     contador += 1 # incrementar para evitar ciclo infinito
16
17 print(" Ciclo terminado.")
18
19 # --- Parte 2: while como acumulador ---
20 print("\n" + "=" * 35)
21 print(" WHILE: SUMA DEL 1 AL 10")
22 print("=" * 35)
23
24 numero = 1
25 suma = 0
26
27 while numero <= 10:
28     suma += numero
29     numero += 1
30
31 print(f" Suma de 1 a 10 = {suma}")
32
33 # --- Parte 3: while True con break ---
34 print("\n" + "=" * 35)
35 print(" WHILE TRUE + BREAK")
36 print(" (máximo 4 intentos)")
37 print("=" * 35)
38
39 intentos = 0
40
41 while True:
42     intentos += 1
43     print(f" Intento numero {intentos}")
44
45     if intentos >= 4:
46         print(" Se alcanza el limite. Saliendo...")
47         break
```

Listing 4: ejemploD.py — while y while True

Parte 1 — Contador. La condición `while contador <= 5` se evalúa *antes* de cada iteración. La línea `contador += 1` es imprescindible: sin ella `contador` nunca

cambiaría y el ciclo correría para siempre (ciclo infinito).

Parte 2 — Acumulador. El patrón `suma += numero` es el acumulador: en cada iteración suma el valor actual de `numero` a `suma`. `numero` se incrementa para avanzar hacia la condición de paro. Cuando `numero` llega a 11, la condición `numero <= 10` es **False** y el ciclo termina.

Parte 3 — While True con break. `while True` crea un ciclo sin condición de paro natural. El **break** dentro del **if** es el único mecanismo de salida. Este patrón es útil cuando la condición de paro se calcula *dentro* del cuerpo del ciclo.

Salida esperada

```
Contador: 1
Contador: 2
Contador: 3
Contador: 4
Contador: 5
Ciclo terminado.

Suma de 1 a 10 = 55

Intento numero 1
Intento numero 2
Intento numero 3
Intento numero 4
Se alcanzo el limite. Saliendo...
```

5 Ejemplo E — Sentencia `match/case`

Objetivo

Usar la sentencia `match/case` de Python 3.10+ para evaluar una variable contra varios valores posibles, como alternativa más legible a una cadena de `if/elif/else`.

Conceptos necesarios

- **match variable:** — evalúa `variable` y la compara con cada `case` en orden.
- **case valor:** — el bloque se ejecuta si `variable == valor`.
- **case _:** — comodín: se ejecuta si *ningún* caso anterior coincidió. Equivale al `else` final.
- Solo el **primer case** que coincida se ejecuta; los demás se ignoran automáticamente (no es necesario `break`).
- Disponible desde **Python 3.10** en adelante.

Enunciado — `ejemploE.py`

Escribe un programa con tres partes que usen `match/case`:

1. **Día de la semana.** Dado un número entero del 1 al 7, imprime el nombre del día correspondiente (1 = Lunes, ..., 7 = Domingo). Para cualquier otro número, imprime `'Numero invalido'`. Prueba con los valores 1 al 8 (usando un ciclo `for`).
2. **Semáforo.** Dada la lista `['rojo', 'amarillo', 'verde', 'azul']`, recorre cada color con un ciclo `for` y usa `match/case` para imprimir la acción correspondiente:

<code>rojo</code>	→ Detente
<code>amarillo</code>	→ Precaucion
<code>verde</code>	→ Avanza
<code>cualquier otro</code>	→ Color no reconocido

3. **Calificación en letra.** Dada la lista `['A', 'B', 'C', 'D', 'F', 'X']`, recorre cada letra con un ciclo `for` y usa `match/case` para imprimir la descripción de la calificación: A = Excelente (90–100), B = Bien (80–89), C = Suficiente (70–79), D = Mínimo (60–69), F = Reprobado (<60), cualquier otra = no reconocida.

Solución — Ejemplo E: match/case

ejemploE.py

```

1  # =====
2  # ejemploE.py - MATCH / CASE - switch-case (nivel básico)
3  # Materia: Algoritmos y Programación en Python
4  # Prof. Dr. Aboud Barsekh-Onji | Universidad Anáhuac México
5  # =====
6
7  # --- Parte 1: Día de la semana ---
8  print("=" * 40)
9  print("  DÍA DE LA SEMANA")
10 print("=" * 40)
11
12 for numero in range(1, 9):          # del 1 al 8 (8 es inválido)
13     match numero:
14         case 1:
15             dia = "Lunes"
16         case 2:
17             dia = "Martes"
18         case 3:
19             dia = "Miercoles"
20         case 4:
21             dia = "Jueves"
22         case 5:
23             dia = "Viernes"
24         case 6:
25             dia = "Sabado"
26         case 7:
27             dia = "Domingo"
28         case _:
29             dia = "Numero invalido"
30     print(f"  Dia {numero}: {dia}")
31
32 # --- Parte 2: Semáforo ---
33 print("\n" + "=" * 40)
34 print("  SEMAFORO")
35 print("=" * 40)
36
37 colores = ["rojo", "amarillo", "verde", "azul"]
38
39 for color in colores:
40     match color:
41         case "rojo":
42             accion = "Detente"
43         case "amarillo":
44             accion = "Precaucion"
45         case "verde":
46             accion = "Avanza"
47         case _:
48             accion = "Color no reconocido"
49     print(f"  {color:<10} -> {accion}")
50
51 # --- Parte 3: Calificación en letra ---
52 print("\n" + "=" * 40)
53 print("  CALIFICACIÓN EN LETRA")
54 print("=" * 40)

```



```

55
56 letras = ["A", "B", "C", "D", "F", "X"]
57
58 for letra in letras:
59     match letra:
60         case "A":
61             descripcion = "Excelente (90 - 100)"
62         case "B":
63             descripcion = "Bien (80 - 89)"
64         case "C":
65             descripcion = "Suficiente (70 - 79)"
66         case "D":
67             descripcion = "Minimo (60 - 69)"
68         case "F":
69             descripcion = "Reprobado (< 60)"
70         case _:
71             descripcion = "Letra no reconocida"
72     print(f" {letra}: {descripcion}")

```

Listing 5: ejemploE.py — match/case básico

Parte 1 — Días de la semana. Cada **case** compara **numero** con un entero literal. El ciclo **for** prueba todos los valores del 1 al 8; el 8 no coincide con ningún **case** concreto y cae al comodín **case _**:. No se necesita **break**: Python sale automáticamente al encontrar el primer caso que coincide.

Parte 2 — Semáforo. La variable **accion** se asigna dentro del bloque **match** y se imprime fuera. El comodín **case _**: captura colores no definidos (aquí, 'azul'). Este patrón es más fácil de leer que cuatro **if/elif** consecutivos.

Parte 3 — Calificación en letra. Los **case** comparan cadenas de texto de un carácter. La 'X' de prueba activa el comodín. Notar que los **case** distinguen mayúsculas y minúsculas: 'a' y 'A' son distintos.

Salida esperada

```

Dia 1: Lunes      Dia 5: Viernes
Dia 2: Martes    Dia 6: Sabado
Dia 3: Miercoles Dia 7: Domingo
Dia 4: Jueves    Dia 8: Numero invalido

rojo      -> Detente
amarillo  -> Precaucion
verde     -> Avanza
azul      -> Color no reconocido

A: Excelente (90 - 100)
B: Bien (80 - 89)
F: Reprobado (< 60)
X: Letra no reconocida

```

5 Resumen comparativo

Ej.	Tema	Funciones/sintaxis clave	Lo que practica
A	for anidados	<code>for</code> , <code>range()</code> , <code>print(end='')</code>	Patrones, tablas, combinaciones
B	random	<code>randint()</code> , <code>choice()</code>	Números y elementos aleatorios
C	Ordenamiento	<code>.sort()</code> , <code>.sort(reverse=True)</code> , <code>sorted()</code>	Ordenar números y palabras
D	while	<code>while cond:</code> , <code>while True:</code> , <code>break</code> , <code>+=</code>	Contador, acumulador, ciclo infinito
E	match/case	<code>match</code> , <code>case</code> , <code>case _:</code>	Despacho por valor, comando

Cuadro 1: Resumen de los cinco ejemplos de nivel básico

Universidad Anáhuac México — Facultad de Ingeniería
 Materia: Algoritmos y Programación en Python — Nivel Básico
 Prof. DSc. Aboud Barsekh-Onji — aboud.barsekh@anahuac.mx
 Ciclo 2025-2026